

Capstone Project 1:

Loan Default Prediction using Machine Learning, FastAPI, MLflow and Azure ML

Submitted by : Abhijeet Kumar

Employee ID : 85000067

Submitted to : Amit Mishra



Abstract:

This project presents an end-to-end machine learning and MLOps solution for predicting loan default risk. The system uses historical borrower and loan-related data to classify whether a borrower is likely to default or not. The workflow includes data preprocessing, exploratory data analysis, feature engineering, class imbalance handling, model training, hyperparameter tuning, model evaluation, model serialization, FastAPI deployment, MLflow experiment tracking, and Azure Machine Learning deployment.

Multiple models were evaluated, including Logistic Regression, Decision Tree, Random Forest, Naive Bayes, and KNN. Random Forest was selected as the final model because it achieved the best overall performance based on ROC-AUC and other metrics.

Problem Statement:

Loan default prediction is important for financial institutions because incorrect lending decisions can cause financial losses. The objective is to build a classification model that predicts whether a borrower will default on a loan.

Target variable:

0 = Non-default

1 = Default

Objectives:

- Load and inspect the loan default dataset.
- Clean missing values and prepare data..
- Encode categorical features and scale numerical features.
- Handle class imbalance using SMOTE.
- Train Logistic Regression, Decision Tree, Random Forest, Naive Bayes, and KNN.
- Tune models using GridSearchCV.
- Evaluate models using Accuracy, Precision, Recall, F1-score, ROC-AUC, and Confusion Matrix.
- Select and save the best model.
- Build a FastAPI /predict endpoint.
- Track experiments using MLflow.
- Deploy model using Azure Machine Learning.

Dataset Description:

The dataset contains borrower and loan-related details such as:

- Loan amount
- Income
- Credit score
- Loan purpose
- Property value
- LTV
- Debt-to-income ratio
- Region
- Credit type
- Default status

Project Structure

```
Loan-Default-Project/  
|  
|—— data/  
|   |—— Loan_Default.csv  
|—— src/  
|   |—— ingest.py  
|   |—— preprocess.py  
|   |—— train.py  
|   |—— evaluate.py  
|—— api/  
|   |—— app.py  
|—— models/  
|   |—— model.pkl  
|   |—— scaler.pkl
```

| └── encoders.pkl

Data Preprocessing :

- Loaded dataset using Pandas.
- Checked shape, data types, and missing values.
- Filled missing numerical values.
- Filled missing categorical values.
- Removed duplicate records.
- Encoded categorical variables.
- Scaled numerical variables.
- Created feature matrix X and target variable y.

Train-Test Split:

The dataset was split into training and testing sets. The training set was used for model learning, while the testing set was used to evaluate performance on unseen data.

Training data: 80%

Testing data: 20%

Models Implemented:

Model	Description
Logistic Regression	Baseline linear classification model
Decision Tree	Tree-based model, easy to interpret
Random Forest	Ensemble tree-based model
Naive Bayes	Probabilistic classification model
KNN	Distance-based classification model

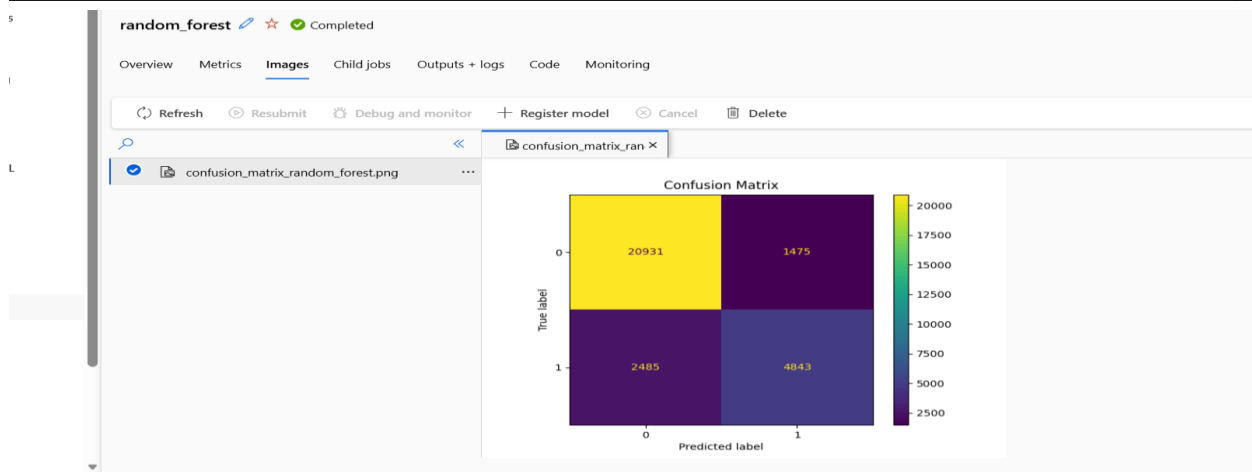
Best Model was found to be: Random Forest

```
=====
FINAL BEST MODEL
=====
Best Model:
random_forest

Best CV ROC-AUC:
0.8573374235576647

Best Params:
{'model__max_depth': 10, 'model__max_features': 'sqrt', 'model__min_samples_leaf': 5, 'model__min_samples_split': 5, 'model__n_estimators': 50}

Best Metrics:
{'accuracy': 0.8668191296159279, 'precision': 0.7665400443178221, 'recall': 0.6608897379912664, 'f1': 0.7098050710831013, 'roc_auc': 0.8618313531943448}
```



Random Forest was selected because it achieved the best ROC-AUC and balanced performance across accuracy, precision, recall, and F1-score. It also handles non-linear relationships better than simpler models and reduces overfitting compared to a single Decision Tree.

Final Model: Random Forest

Metric	Score
Accuracy	0.8668
Precision	0.7665
Recall	0.6609
F1-score	0.7098
ROC-AUC	0.8618
Best CV ROC-AUC	0.8573

Model Serialization :

After selecting Random Forest, the trained model and preprocessing objects were saved for deployment.

Saved artifacts:

- model.pkl
- scaler.pkl
- Encoders.pkl

Model serialization allowed the trained model to be reused by the FastAPI application without retraining.

FastAPI REST API :

FastAPI was used to create a REST API for real-time loan default prediction. The trained Random Forest model was loaded into the API, and a /predict endpoint was created.



The screenshot shows a web browser window with a REST client interface. The top section displays a curl command for a POST request to the endpoint `http://127.0.0.1:8000/predict`. The request body is a JSON object with a "features" array containing 20 numerical values. Below the command, the "Request URL" is shown as `http://127.0.0.1:8000/predict`. The "Server response" section shows a status code of 200. The "Response body" section displays a JSON object with a "prediction" field set to 0. A "Download" button is visible at the bottom right of the response area.

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "features": [
      29921,
      2019,
      1,
      3,
      0,
      1,
      4,
      1,
      0,
      0,
      390590,
      3.125,
      -0.0245,
      129.3,
      180,
      0,
      0,
      0,
      1278000,
      1,
      1,
      1
    ]
  }'
```

Request URL
`http://127.0.0.1:8000/predict`

Server response

Code Details

200

Response body

```
{
  "prediction": 0
}
```

Download

Azure ML Deployment :

Azure Machine Learning was used for cloud deployment and MLOps lifecycle management.

Azure Details :

Resource Group: AbhijeetKr67

Workspace Name: AbhijeetKr67

Model Name: LoanDefaultModel

Endpoint Name: loan-end

Deployment Name: loan-deployment-final-v113

VM Size: Standard_DS1_v2

A smaller VM size was selected because of Azure quota limitations.

The capstone requires Azure ML workspace setup, compute configuration, model registration, managed endpoint deployment, endpoint testing, and monitoring/logging.

Azure Workflow :

- Created Azure ML workspace.
- Configured compute resources.
- Registered the trained model.
- Created managed online endpoint.
- Deployed model to endpoint.
- Tested endpoint using sample input.

loan-endpointxyz123

Details Test Consume Monitoring **PREVIEW** Logs

+ Add deployment ↻ Refresh ✎ Update traffic 🗑 Delete

Endpoint attributes

Service ID

loan-endpointxyz123

Description

Loan default real-time inference endpoint

Provisioning state

○ Updating

Error details

--

Compute type

Managed

Created by

Abhijeet Kumar

Created on

May 11, 2026 7:38 PM

Last updated on

May 11, 2026 7:49 PM

Deployment summary

Live traffic allocation

✔ loan-deployment-xyz1 (100%)

Mirrored traffic allocation

--

Deployment loan-deployment-xyz1

Name

loan-deployment-xyz1

Live traffic

100%

Scoring script

score.py

Provisioning state

✔ Succeeded

Error details

Default Directory > AbhijeetKr67 > Endpoints > loan-endpointxyz123

loan-endpointxyz123

Details **Test** Consume Monitoring **PREVIEW** Logs

Input *

```
{
  "data": [
    {
      "year": 2019,
      "loan_limit": "cf",
      "Gender": "Male",
      "approv_in_adv": "nopre",
      "loan_type": "type1",
      "loan_purpose": "p3",
      "Credit_Worthiness": "l1",
      "open_credit": "nopc",
      "business_or_commercial": "nob/c",
    }
  ]
}
```

Test

jsonOutput

```
{
  "predictions": [
    0
  ],
  "probability_default": [
    0.2812590885337557
  ]
}
```

Conclusion :

This project successfully implemented an end-to-end loan default prediction system. The workflow included data preprocessing, EDA, class imbalance handling, model training, hyperparameter tuning, model evaluation, model serialization, FastAPI deployment, MLflow experiment tracking, and Azure ML deployment.

Five models were evaluated: Logistic Regression, Decision Tree, Random Forest, Naive Bayes, and KNN. Random Forest was selected as the final model because it achieved the best overall performance, with an accuracy of **0.8668** and ROC-AUC of **0.8618**.

The final model was saved as a .pk1 file and integrated into a FastAPI application for real-time prediction. MLflow was used for experiment tracking, and Azure ML was used for model registration and endpoint deployment. This project demonstrates a complete production-ready machine learning and MLOps lifecycle.